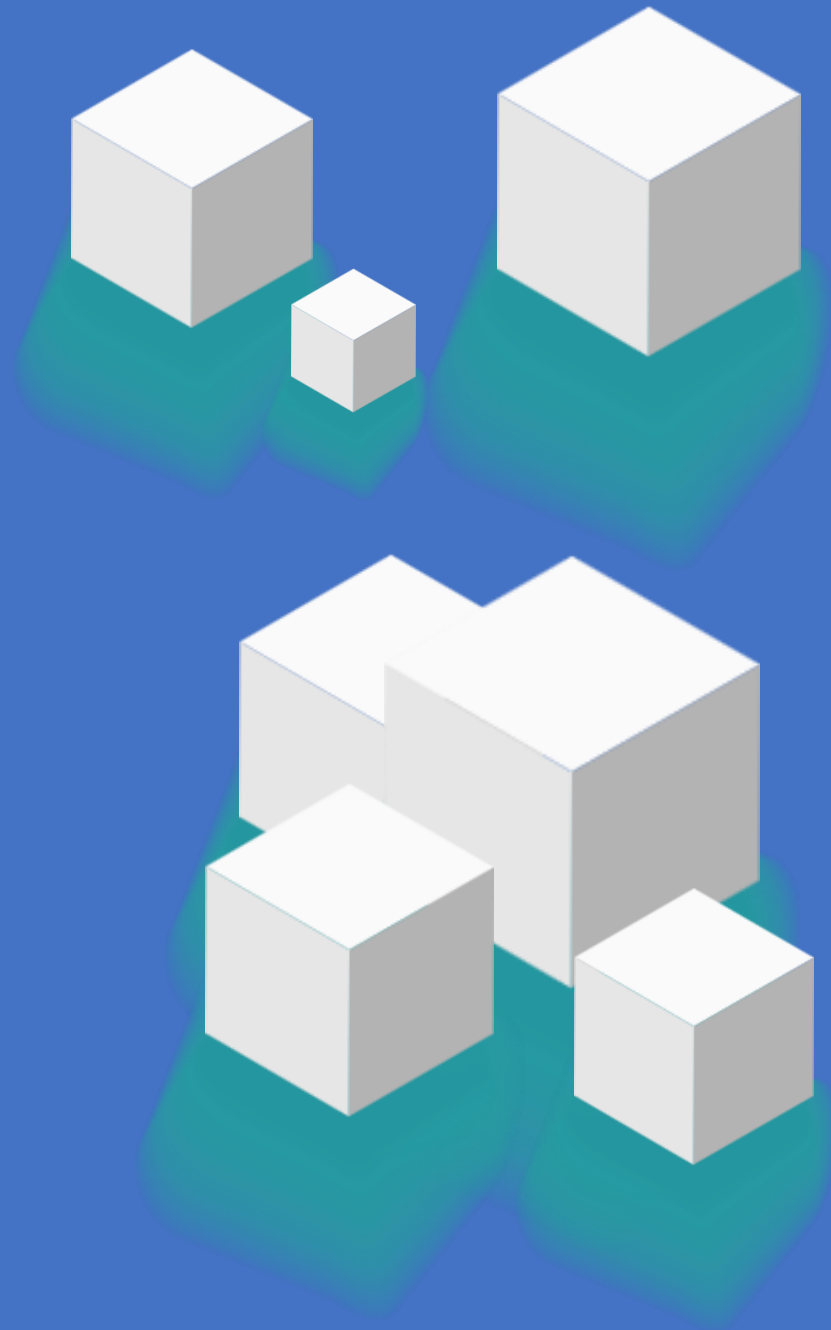
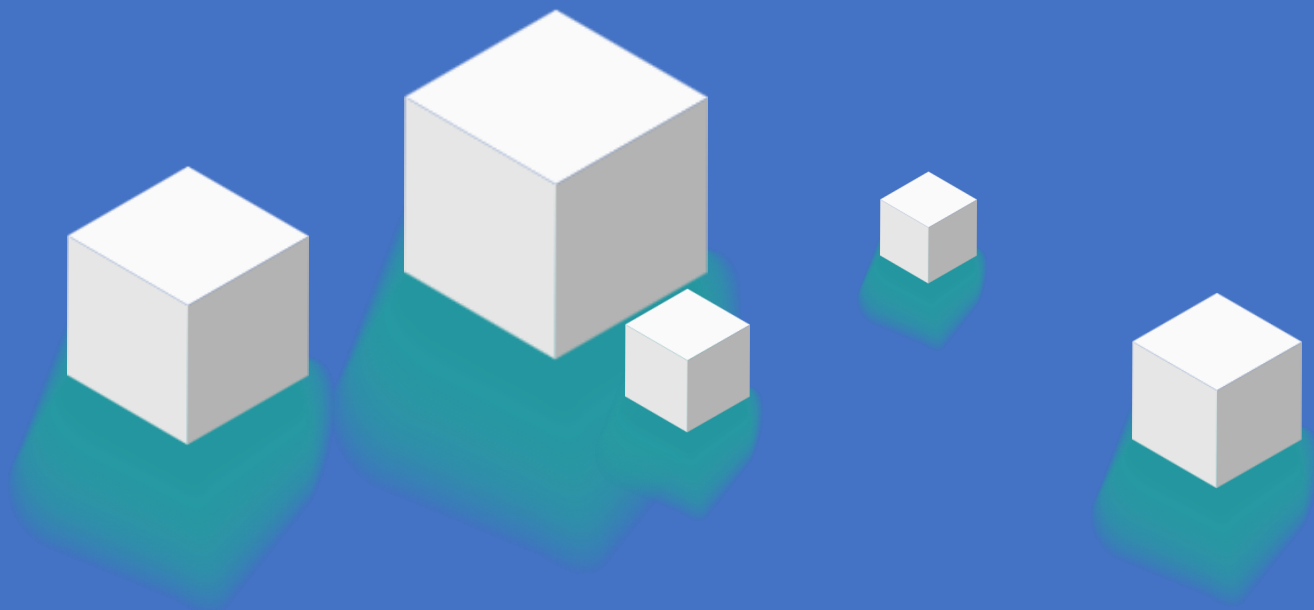


机器学习因子挖掘



>> 今天的学习目标

机器学习因子挖掘

- 因子分类体系

华泰231 因子体系

精简52 因子体系

CASE: 贵州茅台因子分析

- 华泰证券金工研究框架

预处理流水线: MAD去极值

Z-score标准化、行业市值中性化

CASE: 工业级特征工程

CASE: XGBoost 涨跌预测

CASE: LightGBM 对比与调参

因子分类体系

从规则判断 到 模型判断

从规则判断 => 模型判断，我们需要掌握：

- 因子分类体系：价量/动量/波动率/技术指标/均线形态/交互因子
- 华泰金工研究框架：231 个因子的预处理与建模范式
- 工业级特征工程流水线：MAD 去极值 → 行业市值中性化 → Z-score 标准化
- XGBoost 涨跌预测：50+ 因子 + 滚动训练 + 二分类评估
- LightGBM 使用与 Optuna 超参数优化：GOSS/EFB 加速 + TPE 贝叶斯搜索

将传统技术分析的规则判断 => 升级为概率预测

模型输出的上涨概率本身就是一个强大的 Alpha 因子

从规则判断 到 模型判断

Thinking: 传统策略（MACD 金叉买入）和 ML 策略有什么本质区别？

传统技术分析策略的决策逻辑通常是：

```
IF MACD金叉 THEN 买入  
IF RSI > 70 THEN 卖出
```

这种 IF-THEN 规则存在三个根本问题：

- 单一指标无法反映市场全貌
- 阈值（如 RSI=70）是人为设定的，不同股票/不同行情最优阈值不同
- 规则之间无法自动权衡，多个信号冲突时无法决策

从规则判断 到 模型判断

ML 策略的核心转变：从 是/否判断 到 连续概率输出。

维度	传统技术策略	ML 概率策略
信号生成	单指标阈值 (RSI>70 卖出)	50+ 因子综合计算概率
判断方式	是/否二值决策	0~1 连续概率 (如 0.73)
自适应性	固定规则, 需人工调参	自动从数据中学习最优模式
因子交互	手动组合 (如 MACD+KDJ)	树模型自动发现非线性交互
更新机制	不更新或人工回测调整	滚动重训, 自动适应市场变化
输出价值	仅用于买卖信号	概率值本身可作为 Alpha 因子

XGBoost/LightGBM 输出的上涨概率 (0~1) 不仅用于涨跌判断, 还可以作为上涨概率因子, 纳入多因子选股框架, 与其他因子组合使用。

因子分类体系

华泰 12 大类	典型因子
1. 估值因子 (Value)	EP、BP、SP、OCFP、DP
2. 成长因子 (Growth)	营收增长率、扣非净利润增长率、现金流增长率
3. 财务质量因子 (Financial Quality)	ROE、ROA、毛利率、资产周转率
4. 杠杆因子 (Leverage)	资产负债率、现金比率、流动比率
5. 规模因子 (Size)	总市值对数、流通市值对数
6. 动量因子 (Momentum)	过去 1/3/6/12 个月收益率、HAlpha
7. 波动率因子 (Volatility)	1/3/6/12 个月日收益标准差、最高/最低价比
8. 换手率因子 (Turnover)	1/3/6/12 个月换手率
9. 改进的动量因子 (Modified Momentum)	换手率加权收益率（波动与流动性结合）
10. 分析师情绪因子 (Sentiment)	Wind 平均评级、评级变化、目标价偏离度
11. 股东因子 (Shareholder)	户均持股比例、户均持股变化率
12. 技术因子 (Technical)	MACD、DIF、DEA

华泰证券金工团队的 231 因子体系

分析师情绪因子

Wind资讯是中国金融数据服务领域的标杆产品，类似于国际上的Bloomberg（彭博）或Reuters（路透），提供股票、债券、基金、期货、外汇等全品类金融数据

分析师数据：Wind收集并整理了主流券商研究机构对上市公司的盈利预测、投资评级、目标价格等分析师一致预期数据

评级体系：通常采用五档分类——买入/增持/中性/减持/卖出，并提供平均评级指标（例如某股票所有覆盖分析师给出的评级加权平均）

分析师情绪因子（Sentiment）中，Wind数据具体提供：

- **平均评级**：当前分析师一致评级的绝对水平
- **评级变化**：相比前期的评级调整方向（上调/下调）
- **目标价偏离度**：当前股价与分析师一致目标价的偏离程度（反映预期空间）

分析师情绪因子

假设我们正在分析 宁德时代 (300750.SZ) ，收集了近30天内10家券商分析师的评级数据：

券商	最新评级	1个月前评级	目标价(元)	当前股价(元)
中信证券	买入	买入	380	210
中金公司	买入	增持	360	210
华泰证券	增持	增持	340	210
国泰君安	买入	买入	375	210
招商证券	增持	中性	320	210
广发证券	买入	买入	390	210
海通证券	增持	增持	335	210
申万宏源	中性	中性	280	210
中信建投	买入	增持	365	210
天风证券	买入	买入	385	210

Wind评级映射规则（行业通用标准）

买入 = 5分

增持 = 4分

中性 = 3分

减持 = 2分

卖出 = 1分

分数总和 = 5+5+4+5+4+5+4+3+5+5 = 45

平均评级 = 45 ÷ 10 = 4.5分

解读：4.5分介于 增持 (4)和 买入 (5)之间，偏向强烈看好。若行业平均为4.2分，
则该股票分析师情绪高于行业平均。

分析师情绪因子

评级变化 (Rating Change)

上期评级：买入(5)+增持(4)+增持(4)+买入(5)+中性(3)+买入(5)+增持(4)+中性(3)+增持(4)+买入(5) = 42分

上期平均 = $42 \div 10 = 4.2$ 分

变化值：评级变化 = 当前平均 - 上期平均 = $4.5 - 4.2 = +0.3$ 分

量化因子处理：

- 正向变化 (+0.3)：招商证券从 中性 上调至 增持，中信建投从 增持 上调至 买入

信号强度：0.3的差值属于温和上调

- 如果某股票平均评级从2.5分跳升至4.0分（变化+1.5）=> 属于强烈情绪改善，在量化模型中会得到更高的Sentiment得分。

分析师情绪因子

目标价偏离度 (Target Price Deviation)

目标价偏离度 = (Wind一致目标价 - 当前股价) ÷ 当前股价 × 100%

Wind一致目标价 = (380+360+340+375+320+390+335+280+365+385) ÷ 10 = 353元

偏离度 = (353 - 210) ÷ 210 × 100% = +68.1%

+68.1% 代表分析师认为该股票还有68.1%的上涨空间

量化应用:

- 若偏离度 > 50%: 强烈看涨信号 (但需结合基本面判断是否为价值陷阱)
- 若偏离度 < -10%: 分析师看空, 当前股价已高于目标价

因子分类体系

参考华泰证券金工团队的 231 因子体系，精简为 6 大类、52 个核心因子

1. 价量因子（Price-Volume）— 10 个

直接从价格和成交量衍生的基础因子，反映市场交易行为。

因子	含义
ret_1d / ret_3d / ret_5d / ret_10d	1/3/5/10 日收益率
amplitude_5d / amplitude_10d	5/10 日振幅（最高价-最低价 / 均价
vol_ratio_5d / vol_ratio_10d	5/10 日量比（当日成交量 / 均量）
price_volume_corr_10d	10 日量价相关系数
turnover_change_5d	5 日换手率变化

量价关系是技术分析的基石：放量上涨表示多头力量强劲，缩量下跌表示抛压减轻。
华泰研究发现换手率、成交额等价量因子在 SHAP 排名中长期占据前列。

因子分类体系

2. 动量因子（Momentum）— 8 个

衡量价格趋势的持续性和强度，使用 ROC（Rate of Change）计算。

因子	含义
momentum_5d / 10d / 20d / 60d	5/10/20/60 日动量（ROC）
momentum_slope_10d / 20d	动量的斜率（动量的变化率）
momentum_accel_10d / 20d	动量加速度（斜率的变化率）

动量效应是金融学中最持久的异象之一。华泰发现短期反转效应在极端跌幅区间更显著（非线性效应），XGBoost 自动学习了这一非线性关系。

因子分类体系

Thinking: 怎么理解动量ROC，动量的斜率，以及动量加速度？

速度 → 加速度 → 加加速度的层级关系。

动量是车速，动量斜率是油门深浅（加速度），动量加速度是踩油门的速度（加加速度）。

交易日	价格	动量 (5日ROC)	动量斜率	动量加速度
Day 1	10元	-	-	-
Day 2	11元	-	-	-
Day 3	12元	-	-	-
Day 4	13元	-	-	-
Day 5	14元	$(14-10)/10=40\%$	-	-
Day 6	16元	$(16-11)/11=45.5\%$	+5.5% (斜率1)	-
Day 7	19元	$(19-12)/12=58.3\%$	+12.8% (斜率2)	+7.3% (加速)
Day 8	21元	$(21-13)/13=61.5\%$	+3.2% (斜率3)	-9.6% (减速)

因子分类体系

3. 波动率因子 (Volatility) — 6 个

衡量价格波动的剧烈程度，低波动股票往往有更高的风险调整收益。

因子	含义
atr_norm_14	14 日标准化 ATR（波动率的无量纲度量）
hist_vol_10d / 20d / 60d	10/20/60 日历史波动率（年化）
vol_change_10d / 20d	波动率变化率

低波动异象 (Low Volatility Anomaly) 是经典因子之一。
波动率因子也用于度量市场恐慌程度和定价风险溢价。

因子分类体系

Thinking: 怎么理解低波动异象, 以及 高风险高收益?

低波动异象是金融学最著名的异象之一: 涨得慢的股票, 长期性价比 (夏普比率) 反而比暴涨暴跌的妖股高。
这确实和教科书上 高风险高收益 矛盾, 但实证数据如此。

比如: 2025年两只股票:

股票	特征	全年走势	年化收益	最大回撤	夏普比率
长江电力 (低波)	每天波动 $\pm 1\%$	稳步上涨, 偶尔回调 3%	15%	-8%	1.2
某概念妖股 (高波)	每天波动 $\pm 8\%$	连续涨停后连续跌停	20%	-60%	0.3

虽然妖股绝对收益略高 (20% vs 15%) , 但你要承担 60% 的回撤风险。
而长江电力单位风险带来的收益是妖股的 4 倍。

这就是 风险调整收益 (夏普比率) 的核心: 投资者追求的是 睡得好觉的收益

因子分类体系

Thinking: 为什么高波动股票反而性价比低? (行为金融学解释)

如果教科书说高风险高收益, 那高波动股票应该有高收益才对, 但现实是:

彩票偏好: 散户喜欢买高波动股票, 过度追捧导致价格虚高, 长期收益反而被压低

杠杆限制: 机构不愿重仓高波动股票 (风控限制), 散户成为高波动股票的主要买家

追涨杀跌: 高波动股票往往伴随高换手, 摩擦成本吃掉收益

=> 市场并非完全理性, 高波动股票往往被过度定价, 低波动股票被忽视定价。

散户接力: 机构不玩的高波动股票, 正好成为散户赌博的乐园 (涨停敢死队), 散户的购买力推高了高波动股票的价格

边际定价: 股票价格由边际买家决定。当机构因风控退出后, 散户成为高波动股票的主要买家, 他们的非理性追捧导致高估

因子分类体系

4. 技术指标因子 (Technical) — 12 个

通过 TA-Lib 计算的经典技术分析指标，捕捉超买超卖和趋势强度信号。

因子	含义
rsi_14 / rsi_6	相对强弱指标 (超买/超卖)
adx_14	平均方向指数 (趋势强度)
macd_hist / macd_signal / macd_dif	MACD 柱状图 / 信号线 / DIF
bbands_position	布林带位置 (0~1)
kdj_k / kdj_d	KDJ 随机指标
cci_14	商品通道指标
willr_14	威廉指标
obv_slope_10d	OBV 斜率 (量能趋势)

华泰发现 RSI 的超买超卖阈值并非固定的 30/70，
XGBoost 自动学习了不同市场状态下的最优分割点。

因子分类体系

5. 均线与形态因子（MA Pattern） — 10 个

均线偏离度和 K 线形态特征，反映技术面的多空力量对比。

因子	含义
ma5_bias / ma10_bias / ma20_bias / ma60_bias	收盘价偏离均线的比率
ma_bull_score	多头排列评分 (0~1), 6 个条件加权
upper_shadow_ratio / lower_shadow_ratio	上/下影线比率
body_ratio	实体比率
new_high_20d / new_low_20d	20 日新高 / 新低标记

因子分类体系

6. 交互因子 (Interaction) — 6 个

多个因子的交叉组合，捕捉因子间的非线性关系。

因子	组合方式	金融含义
mom_vol_cross	动量 × ATR	低波动环境中动量效应更强
adx_rsi_cross	ADX × (RSI-50)/50	趋势强度调节超买超卖信号
vol_ratio_mom_cross	量比 × 动量	放量突破 vs 缩量突破
rsi_bbands_cross	RSI × 布林带位置	超买信号在布林上轨更有效
macd_adx_cross	MACD × ADX	MACD 在强趋势中更可靠
vol_mom_accel_cross	波动率 × 动量加速度	波动放大时趋势加速更显著

部分因子存在强交互作用，这也是我们在体系中加入 交互因子 大类的原因。
线性模型难以捕捉这些交互效应，树模型天然适合处理。

行业因子

行业分类也是重要的 Alpha 来源。在华泰 231 因子体系中，行业因子可以 One-Hot 编码进入模型：

```
# 行业哑变量构造示例
industry_map = {
    '600519.SH': '食品饮料',
    '000858.SZ': '食品饮料',
    '601318.SH': '非银金融',
    '600036.SH': '银行',
}
industry_dummies = pd.get_dummies(df['industry'], prefix='ind')
# 结果: ind_食品饮料=1, ind_非银金融=0, ind_银行=0 ...
```

行业因子的三个作用：

- **控制行业暴露**：避免模型过度集中在某些行业
- **行业轮动信号**：不同行业在不同经济周期表现不同
- **因子中性化**：剔除因子中的行业效应，得到纯因子收益

基本面因子

华泰核心发现：

- 价量因子重要性 > 基本面因子。
- 在 SHAP 特征重要性排序中，换手率、成交额、波动率长期占据前列，
- PE、ROE 排名靠后 => 这与 市场短期由交易行为驱动的直觉一致。

因子	含义	特征
pe_ratio	市盈率（价格/每股收益）	估值因子，低频更新
roe_factor	净资产收益率	盈利质量因子
gross_margin_factor	毛利率	盈利能力因子
debt_ratio_factor	资产负债率	财务风险因子

CASE: 贵州茅台因子分析

TO DO: 搭建因子分析代码，并对 贵州茅台 进行分析 => 理解因子是什么，好不好用

在单只股票（默认 600519.SH）的时间序列上：

1. 枚举 feature_engine 中的因子分类体系（FACTOR_TAXONOMY）。
2. 从 MySQL 拉日 K，调用 calc_features 生成技术面因子。
3. 可选：拉财务表，调用 calc_fundamental_features 合并基本面因子。
4. 用 pd.get_dummies 演示行业 One-Hot（小表示例，非全市场）。
5. 对每个因子与次日收益率算 RankIC（Spearman），汇总排名与按类别统计。

CASE: 贵州茅台因子分析

日K (DataFrame) --calc_features--> 含 52+ 技术列的 df + calc_fundamental_features(fin_df, ...) --> 增加基本面列

```
fwd_ret_1d = close.pct_change(1).shift(-1) # 标签：下一交易日收益
```

对每个因子列 col: RankIC(col, fwd_ret_1d)

代码功能：

- data_loader.load_stock_data：按股票代码、日期区间读日 K（OHLCV 等）
- data_loader.load_financial_data：读财务宽表，供基本面因子对齐
- feature_engine.FACTOR_TAXONOMY：六大类因子名与列表（这里用 52 个技术因子定义）
- feature_engine.calc_features：由单股 OHLCV 计算全部技术因子列
- feature_engine.calc_fundamental_features：将财务数据按报告期对齐到日频并生成 PE/ROE 等列
- feature_engine.get_all_feature_cols：返回技术因子列名列表，用于筛选 df 中实际存在的列

1-贵州茅台因子分析.py

CASE: 贵州茅台因子分析

核心函数: calc_rank_ic

```
def calc_rank_ic(factor_values, forward_returns):  
    valid = pd.DataFrame({  
        'factor': factor_values,  
        'fwd_ret': forward_returns  
    }).dropna()  
    if len(valid) < 30:  
        return np.nan  
    ic, _ = spearmanr(valid['factor'], valid['fwd_ret'])  
    return ic
```

- 使用 Spearman 相关系数：等价于对因子值与 forward return 先做秩（rank）再算 Pearson 相关，即常见的 RankIC。
- dropna(): 任一侧缺失则丢弃该日样本。
- 有效样本少于 30 天不计算（避免统计不稳定），返回 nan。
- 未做截面：这里是单股上「因子序列 vs 次日收益序列」的时序 IC，与多股截面 IC 定义不同

CASE: 贵州茅台因子分析

Thinking: RankIC 是什么?

IC: 看「因子」和「未来收益」是否统计上相关, 绝对值越大说明关系越强。

Rank: 不纠结数值本身多大, 只看排序=> 减少极端值、量纲影响。

Spearman = 先 rank 再 Pearson => 是数学定义层面的等价关系

对「因子」和「次日收益」各算一列名次, 再对这两列名次算 Pearson 相关, 结果就等于 Spearman 相关。

所以 `spearmanr(因子, fwd_ret)` 算的就是业内常说的 RankIC。

CASE: 贵州茅台因子分析

RankIC 检验标准

```
RankIC = Spearman(rank(factor_t), rank(return_t+1))
```

$|IC| > 0.05$: 较强的预测能力

$|IC| > 0.03$: 有效因子

$|IC| > 0.02$: 弱有效因子

$|IC| < 0.02$: 无效因子

CASE: 贵州茅台因子分析

数据：贵州茅台 600519.SH，2023-01-03 ~ 2025-12-31，723 个交易日。

共 56 个因子（52 技术 + 4 基本面）完成 RankIC 检验。

这 4 个基本面因子在 calc_fundamental_features 里写死为下面四列（财务字段按报告期 ffill 对齐到日频）：

因子列名	含义	数据来源
pe_ratio	市盈率（估值）	close / eps, eps 来自财务表
roe_factor	ROE（盈利质量）	财务表列 roe
gross_margin_factor	毛利率	财务表列 gross_margin
debt_ratio_factor	负债率	财务表列 debt_ratio

CASE: 贵州茅台因子分析

TOP-10 最强预测因子:

排名	因子	类别	RankIC
1	macd_dif	技术指标	-0.1006
2	mom_vol_cross	交互因子	-0.098
3	adx_rsi_cross	交互因子	-0.0933
4	momentum_20d	动量因子	-0.0908
5	ma20_bias	均线形态	-0.0906
6	rsi_14	技术指标	-0.0888
7	momentum_10d	动量因子	-0.0861
8	ret_10d	价量因子	-0.0861
9	macd_signal	技术指标	-0.0814
10	price_volume_corr_10d	价量因子	-0.0812

CASE: 贵州茅台因子分析

因子质量分布：

等级	标准	数量
较强	$ IC \geq 0.05$	24
有效	$ IC \geq 0.03$	9
弱有效	$ IC \geq 0.02$	5
无效	$ IC < 0.02$	18

各类因子平均 $|IC|$ 排名：

类别	平均 $ IC $	最大 $ IC $	因子数
交互因子	0.081	0.098	6
技术指标	0.0554	0.1006	12
动量因子	0.0469	0.0908	8
均线形态	0.0463	0.0906	10
基本面	0.0453	0.0631	4
价量因子	0.0335	0.0861	10
波动率	0.0107	0.0191	6

CASE: 贵州茅台因子分析

通过运行 贵州茅台因子分析，可以看到：

- 交互因子类平均 $|IC|$ 最高 (0.081)，验证了华泰 因子间非线性组合比单因子更有效 的结论
- 所有强因子的 IC 均为负值，说明贵州茅台在 2023-2025 年存在明显的反转效应（涨多了容易跌回来）
- 波动率因子平均 $|IC|$ 最低 (0.011)，波动率本身对单只股票次日涨跌的直接预测力较弱
- 56 个因子中有 24 个达到较强 级别 ($|IC| \geq 0.05$)，说明因子体系对茅台有较好的覆盖度
- 华泰 价量因子 > 基本面因子 结论得到验证：基本面因子平均 $|IC| = 0.045$ ，略低于动量/技术类

打卡：自选股因子分析



针对你的股票池，进行因子分析：

- 从 MySQL 拉日 K，调用 calc_features 生成技术面因子。
- 拉财务表，调用 calc_fundamental_features 合并基本面因子。
- 查看对应 feature_engine 中的因子 (FACTOR_TAXONOMY)
- 对每个因子与次日收益率算 RankIC (Spearman)，汇总排名与按类别统计。

通过因子分析，你的自选股有怎样的规律？

华泰证券金工研究框架

华泰证券金工研究框架

华泰金工团队的 XGBoost 选股模型是 A 股量化领域的经典研究，其核心流程：

231 个因子 → MAD 去极值 → 行业市值中性化 → Z-score 标准化 → XGBoost 训练 → SHAP 解释

MAD (Median Absolute Deviation, 中位数绝对离差)：

衡量数据离散程度的稳健统计量，计算公式为各数据点与中位数偏差绝对值的中位数。

=> 相比标准差，MAD 基于中位数计算，对极端值完全不敏感。

预处理流水线：MAD去极值

Step 1: MAD 去极值

Thinking: 为什么不用经典的 3-sigma 法?

3-sigma 法基于均值和标准差，而均值和标准差本身就容易受极端值影响，
形成 极端值影响统计量 → 统计量放宽截断边界 → 极端值保留 的恶性循环。

MAD (Median Absolute Deviation) 基于中位数，对极端值完全不敏感：

```
median = series.median()
mad = (series - median).abs().median()
upper = median + 5 * 1.4826 * mad
lower = median - 5 * 1.4826 * mad
series = series.clip(lower=lower, upper=upper)
```

1.4826 的含义：正态分布下， $\text{MAD} \times 1.4826 \approx \text{标准差}$ 。

乘以 5 表示保留约 5 个标准差以内的数据，比 3-sigma 更宽容，
只截断真正的极端异常值。

Thinking: 这里的5是固定的倍数，还是可以变化的?

预处理流水线：MAD去极值

5 倍是经验值（可调整为 3-5 倍），1.4826 是常数（正态分布下使 $MAD \approx$ 标准差）。

比如针对数据：[1, 2, 3, 4, 100]（100 是异常值）

3-sigma方法：mean=22, std≈44.3

截断边界： $22 \pm 3 \times 44.3 = [-110.9, 154.9]$

=> 100 被保留（均值和标准差都被异常值拉大了）

MAD：median=3, mad=1

截断边界： $3 \pm 5 \times 1.4826 \times 1 \approx [-4.4, 10.4]$

=> 100 被截断（中位数和 MAD 不受极端值影响）

3-sigma 的尺子本身会被异常值扭曲，导致边界过宽；
MAD 的尺子基于中位数，始终稳定，能识别真正的极端值。

预处理流水线：Z-score标准化

Step 2: Z-score 标准化

将所有因子统一到均值为 0、标准差为 1 的尺度：

```
mean = series.mean()
std = series.std()
series = (series - mean) / std
```

Thinking：为什么需要标准化？

- RSI 的范围是 0~100，MACD 的范围可能是 -5~5
- 不同量纲的因子直接输入模型，大数值因子会主导分裂
- 标准化后所有因子在同一尺度上竞争，模型选择更公平

预处理流水线：行业市值中性化

Step 3: 行业市值中性化

核心逻辑：把因子值拆解成 行业共性 + 市值共性 + 个股个性，然后扔掉共性，只保留个性。

Thinking：为什么要中性化？——先看偏差

假设你用 BP（市净率倒数） 选股：

股票	行业	市值	BP值	问题
工商银行	银行	大	0.8	银行本身就高BP，不是它便宜，是行业特性
招商银行	银行	大	0.7	同上
宁德时代	新能源	大	0.2	科技行业BP天然低
某小银行	银行	小	0.9	小市值往往估值低

如果不中性化：银行板块会霸榜（BP天然高），小盘股被系统性压低（小市值因子干扰）
=> 你选出的不是 便宜的好公司，而是 银行+小市值

预处理流水线：行业市值中性化

Thinking：如何对行业市值进行中性化处理？

回归法拆解——剥离共性

回归方程本质是在问：这只股票的BP，多少是因为它是银行？多少因为它市值大？

剩下多少才是它真正的估值便宜？

BP = β_0 (基础值)
+ $\beta_{\text{bank}} \times 1$ (如果是银行，加多少)
+ $\beta_{\text{mktcap}} \times \ln(\text{市值})$ (市值每大1%，BP变化多少)
+ ε (残差，真正的个股估值)

计算过程：

- 拟合共性：用全市场数据算出 银行的平均BP溢价、市值对BP的影响系数
- 预测BP：根据个股的行业和市值，算出它 应该有的BP (即模型预测值)
- 求残差：实际BP - 应该有的BP = 个股真实估值溢价

预处理流水线：行业市值中性化

通过回归法消除行业 and 市值对因子的影响：

```
# factor =  $\beta_0$  +  $\beta_{\text{industry}}$  × industry_dummies +  $\beta_{\text{mktcap}}$  × ln(mktcap) + residual  
# 残差 residual 即为中性化后的因子值  
  
from sklearn.linear_model import LinearRegression  
  
X = pd.concat([industry_dummies, mktcap_log.to_frame()], axis=1)  
model = LinearRegression().fit(X, factor_values)  
neutralized_factor = factor_values - model.predict(X) # 残差
```

中性化的效果：消除行业间的因子均值差异，
使因子反映的是个股相对于行业的超额信息，而非行业层面的系统性差异。

预处理流水线：行业市值中性化

举例：针对银行股进行中性化，假设回归结果：

银行行业的BP比基准高 +0.5

市值每增加1%，BP降低 -0.1

股票	实际BP	预测BP（行业+市值）	残差 (中性化后)	含义
工商银行	0.8	0.5（银行） + 0.3（大市值修正） = 0.8	0	符合行业/市值定位，无超额优势
招商银行	0.7	0.5（银行） + 0.3 = 0.8	-0.1	相比同梯队银行，估值偏贵
某小银行	0.9	0.5（银行） + 0.5（小市值修正） = 1.0	-0.1	看似BP高，小市值后偏贵
宁德时代	0.2	-0.1（非银行） + 0.2（大市值） = 0.1	+0.1	在科技行业中，估值相对偏低（真正便宜）

关键变化：中性化后，宁德时代从BP垫底变成相对便宜，而工商银行从BP最高变成平庸。
此时因子反映的是跨行业的真实估值差异，而非行业偏见。

Summary (行业市值中性化)

假设最终要选出低估值股票：

- 中性化前：前10名全是银行+小盘股（伪便宜）
- 中性化后：各行业都有代表，选出的是 银行中的便宜货 和 新能源中的便宜货，分散且真实反映alpha

=> 华泰231因子体系中，行业市值中性化作为标准预处理步骤的原因，

确保因子暴露的是pure alpha，而非beta伪装。

预处理流水线

完整流水线代码：

```
from feature_engine import calc_features, preprocess_features, neutralize

# 1. 计算 50+ 技术因子
df = calc_features(df)

# 2. MAD 去极值 + Z-score 标准化
feature_cols = get_all_feature_cols()
df = preprocess_features(df, feature_cols, method='mad')

# 3. 行业市值中性化（可选，多股票截面场景下使用）
factor_neutralized = neutralize(
    factor_series=df['momentum_20d'],
    industry_dummies=industry_dummies,
    mktcap_log=mktcap_log,
)
```

华泰证券金工研究框架

通过分析，可以得出三大结论：

结论1：价量类因子重要性 > 基本面因子

- SHAP 排名中，换手率、成交额、波动率长期占据前列
- 基本面因子（PE、ROE）排名靠后
- 市场短期由交易行为驱动

结论2：XGBoost 以非线性逻辑使用因子

- 市值因子不是简单的小盘股溢价，而是存在 U 型关系
- 反转因子的短期反转效应在极端跌幅区间更显著
- RSI 超买超卖阈值并非固定 30/70，XGBoost 自动学习最优分割点

华泰证券金工研究框架

结论3：部分因子存在强交互作用

- 动量 $\times$ 波动率：低波动环境中动量效应更强
- ADX $\times$ RSI：趋势强度调节超买超卖信号的有效性
- 成交量 $\times$ 动量：放量突破比缩量突破的持续性更强

线性模型难以捕捉上述非线性和交互效应，
树模型（XGBoost/LightGBM）天然适合处理因子间的非线性交互。
因子工程 + 机器学习 = 现代量化选股的核心范式

CASE: 工业级特征工程

TO DO: 针对 5 只股票（茅台/中芯国际/平安银行/纳指ETF/宁德时代），
进行特征工程流水线（7 个步骤）

步骤	函数调用	作用
Step 1	load_stock_data() × 5次	逐只加载日 K 线（茅台/中芯/平安/纳指ETF/宁德时代）
Step 2	calc_features(df)	对每只股票计算 52 个技术因子
Step 3	calc_fundamental_features()	从财务数据表补充 PE/ROE/毛利率/负债率
Step 4	手动 One-Hot 编码	构造行业哑变量（5 只股票 × 5 个行业）
Step 5	preprocess_features(method='mad')	MAD 去极值 + Z-score 标准化
Step 6	neutralize()	对 momentum_20d 做行业市值中性化（回归取残差）
Step 7	corr() 矩阵分析	识别 corr > 0.8 的冗余特征对

CASE: 工业级特征工程

观察重点:

- 去极值前后的 min/max 变化了多少?
- 标准化后均值是否接近 0、标准差接近 1?
- 中性化后各行业的因子均值是否趋于一致?
- 有多少对冗余特征? 哪些因子高度相关?

CASE: 工业级特征工程

运行结果：

Step 1 数据加载： 5 只股票各 723 个交易日（2023-01-03 ~ 2025-12-31）

Step 5 预处理前后对比：

因子	预处理前 min	预处理前 max	预处理后 min	预处理后 max	预处理后 mean	预处理后 std
momentum_20d	-20.95	126.8	-3.73	3.94	0	1
rsi_14	16.01	92.55	-2.98	3.42	0	1
macd_hist	-16.16	50.23	-3.33	4.59	0	1

Step 6 中性化效果： 中性化后各行业的 momentum_20d 均值全部趋近于 0， 行业间因子差异被消除。

Step 7 相关性分析：

- 高相关特征对（|corr| > 0.8）： 67 对
- 冗余特征： 24 个， 保留后 28 个（原 52 个）
- 典型冗余对： ret_5d 与 momentum_5d (corr=1.0)、 macd_hist 与 macd_adx_cross (corr=0.96)

CASE: 工业级特征工程

结论洞察:

1. MAD 去极值有效截断了极端值 (momentum_20d 从 $[-20.9, 126.8]$ 压缩到 $[-3.7, 3.9]$) , 且保留了正常数据的分布形状
2. 标准化后所有因子均值 ≈ 0 、标准差 ≈ 1 , 不同量纲的因子统一到同一尺度
3. 52 个因子中有 24 个冗余 (存在高度相关性) , 实际有效的独立因子约 28 个。
=> 这提示在后续建模中可考虑降维
4. 行业市值中性化是多股票截面场景下的必要步骤, 单股票场景 (如脚本 1) 不需要

机器学习基础

Using data to solve problems



AI的本质

AI就是利用数据，解决问题

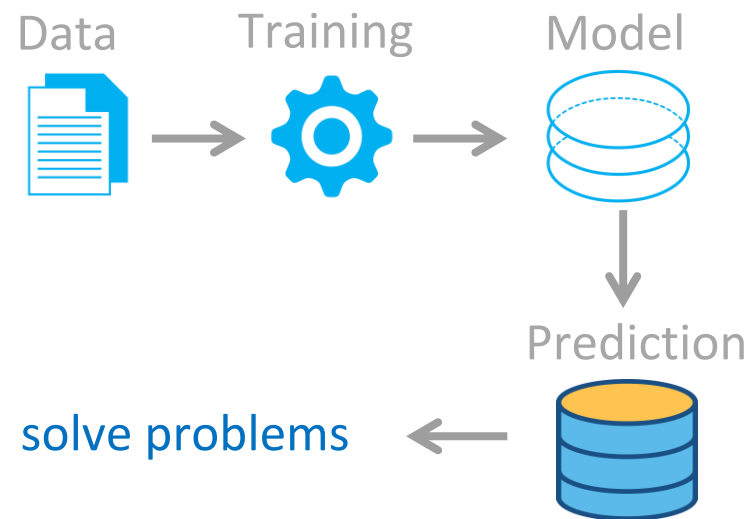
Using data to solve problems

Training

Prediction

Using data

solve problems



训练阶段：通过对数据的训练，创建一个预测模型并对其进行微调。

模型生成：预测模型可以从这些数据背后找出答案来，帮我们解决某个问题。

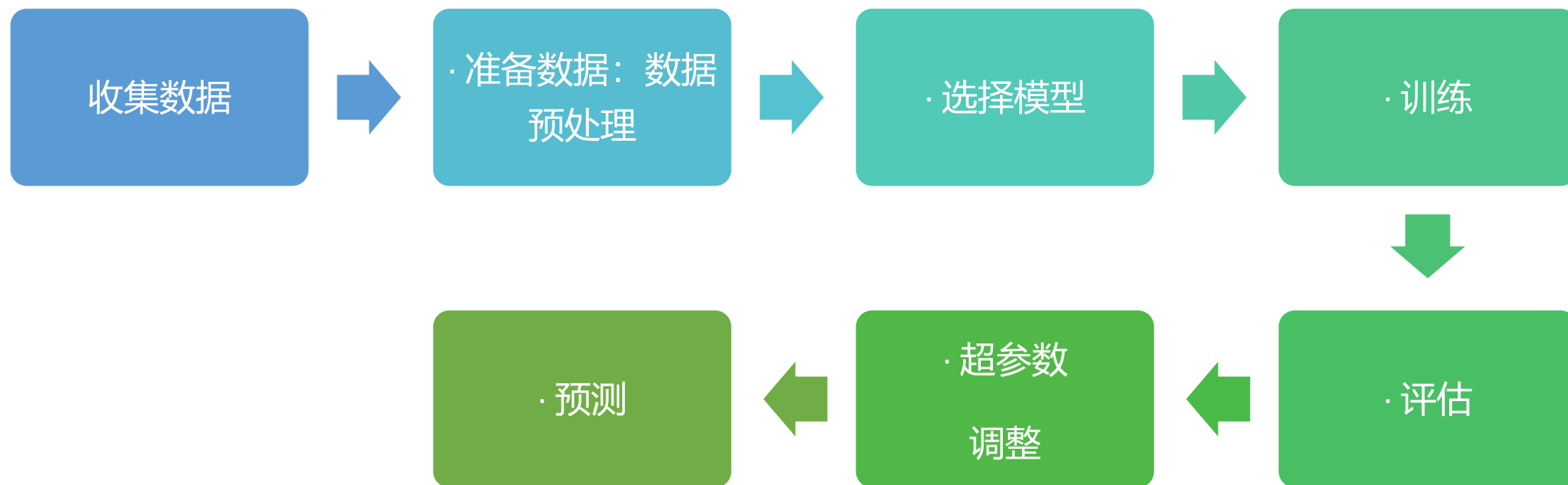
预测阶段：通过测试集完成模型评估，从而了解模型在测试集中的有效性。

过程中，预测模型会被不断改进和使用。

机器学习的步骤

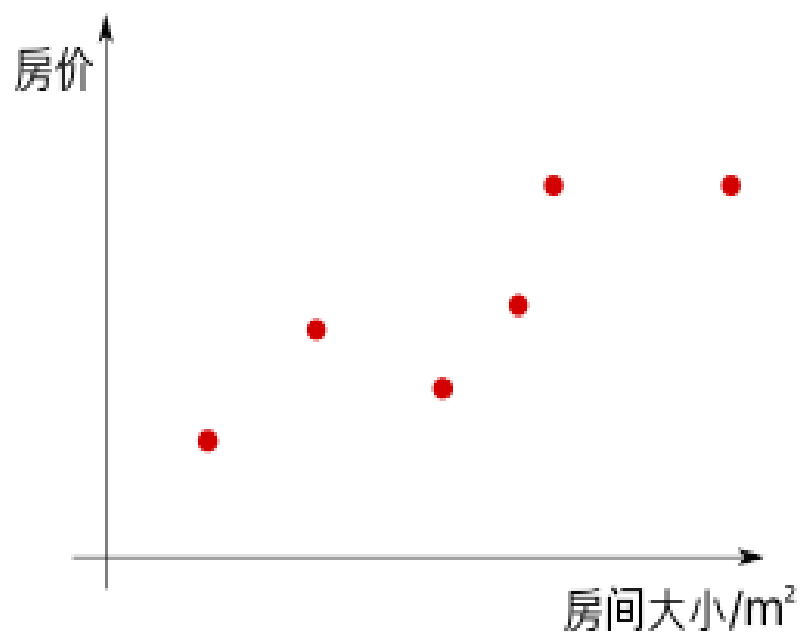
- Thinking: 如何预测房价?

机器学习的7个步骤



机器学习的步骤

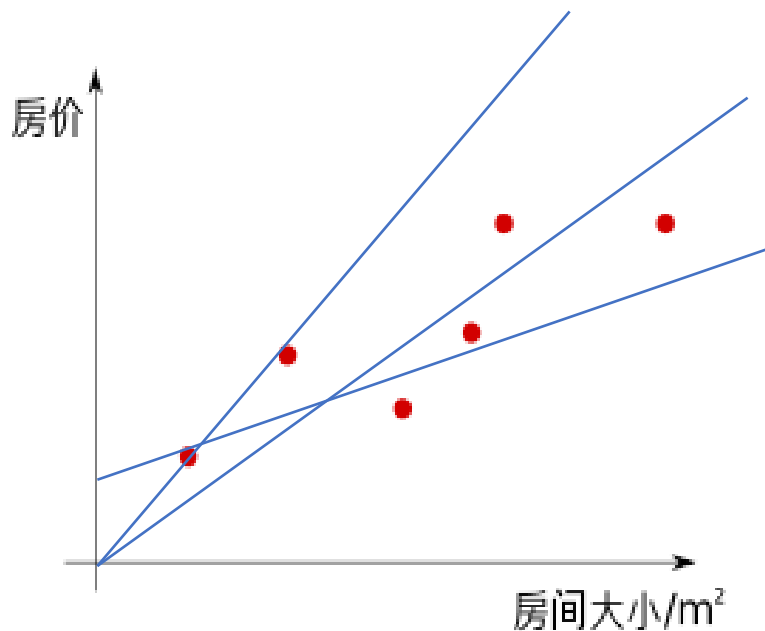
- Thinking: 如何预测房价?



房间大小x	房价y
50	82
80	118
100	172
200	302
.....	

机器学习的训练过程

- 训练是机器学习的主要步骤
- 针对预测房价这个例子，我们可以采用简单的线性模型： $y = w * x + b$



机器学习的训练过程

- 在机器学习中，我们有很多特征，基于这些特征，我们需要训练在Model中的权重 w
- 这些特征值构成的矩阵，称之为权重矩阵 weights
- 同时，还存在偏差，称之为 biases

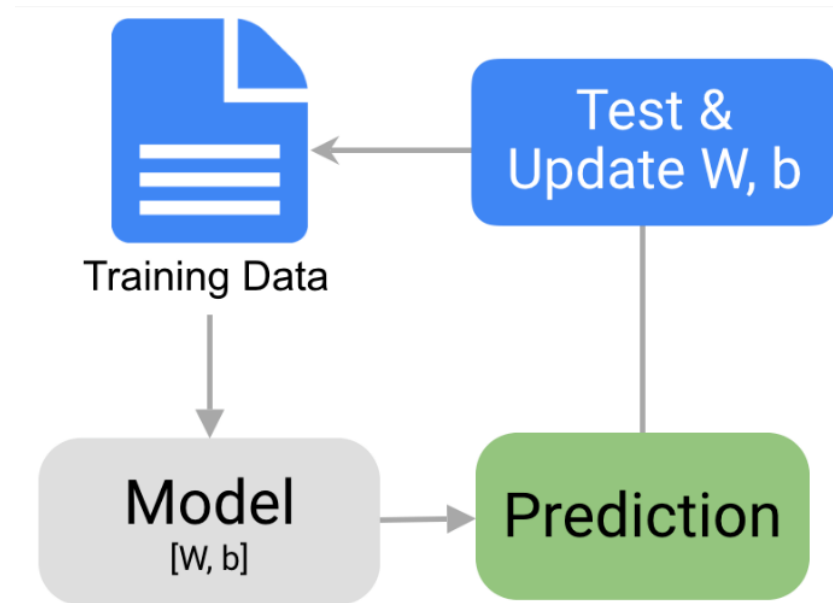
房间大小	区域	周围绿化	周边配套	房型	房价y
50	海淀	A	A	style1	82
80	通州	B	A	style1	118
100	朝阳	C	B	style2	172
200	海淀	C	C	style3	302
.....					

$$f(x) = w_1x_1 + w_2x_2 + \dots + w_dx_d + b$$

$$f(x) = \mathbf{w}^T \mathbf{x} + b$$

机器学习的训练过程

- 机器学习的过程，就是在搜索空间中对 W 和 b 进行搜索的过程，使得模型的准确率达到某个标准
- 一个训练步骤(training step)，称之为一次迭代。目的在于更新权重和变量
- 通过多次迭代，模型中的参数不断进行更新。就好像是在数据中进行线性拟合
- 当完成训练时，可以使用模型对房价进行预测



机器学习的模型选择

- Thinking: 什么是回归问题, 什么是分类问题?
- Thinking: 什么是线性回归, 什么是逻辑回归?

机器学习的模型选择

- 判断一个问题是分类，还是回归：
输出的数据类型：离散 or 连续

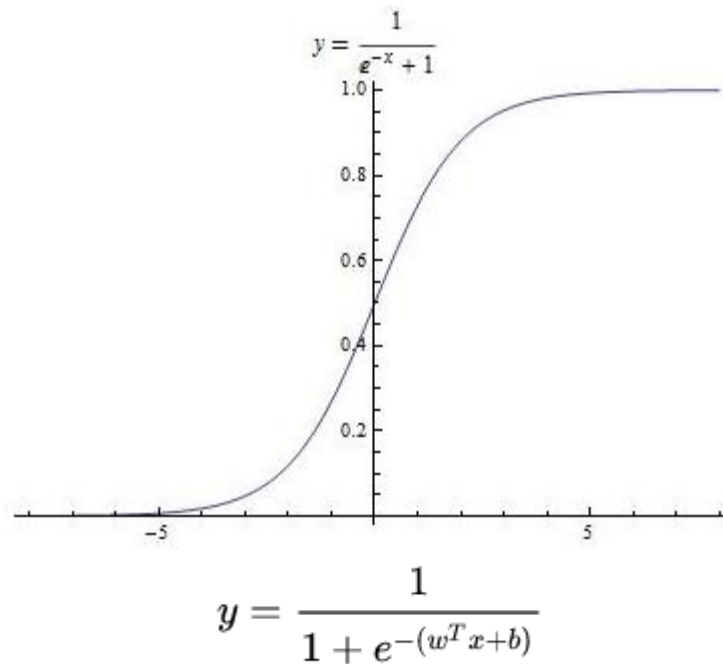
- 线性回归：

$$f(x) = w_1x_1 + w_2x_2 + \dots + w_dx_d + b$$

$$f(x) = \mathbf{w}^T \mathbf{x} + b$$

- 逻辑回归：

使用sigmoid函数，实际上是分类算法



机器学习的模型选择

- Thinking: 如何判断杯子里盛的是水, 还是饮料?

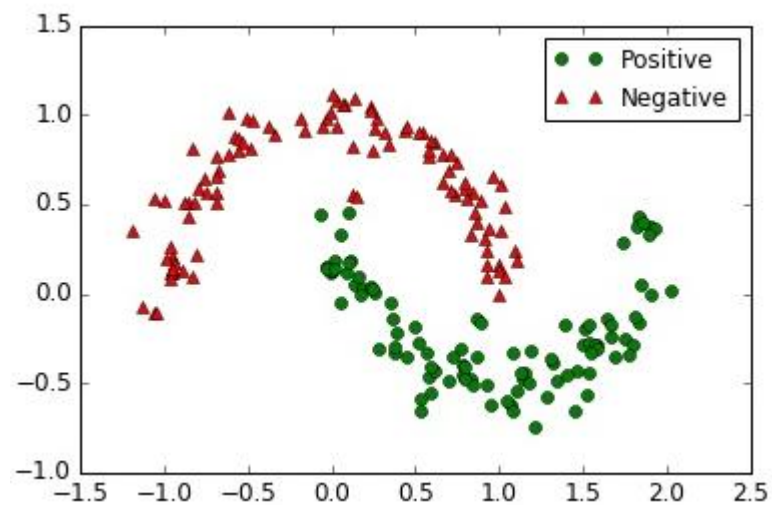
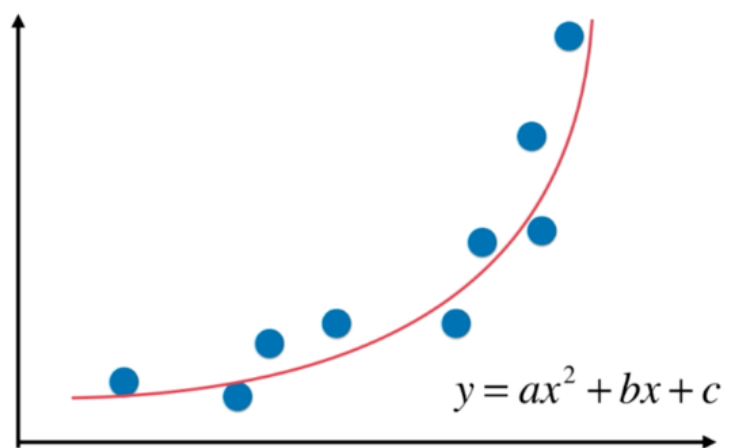


机器学习的模型选择

Color	Sugar	Classification
252	0.1%	water
210	4%	beverage
150	8%	beverage
250	0.5%	water
.....		

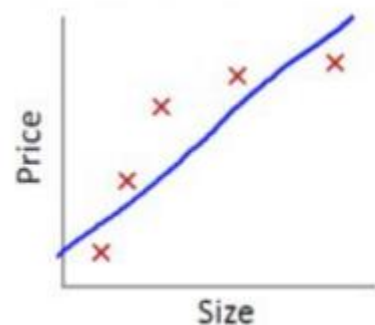
可见光的波长为400 ~ 760nm，白色是包含光谱中所有颜色的集合
因此采用Color这里采用颜色值

机器学习的模型选择

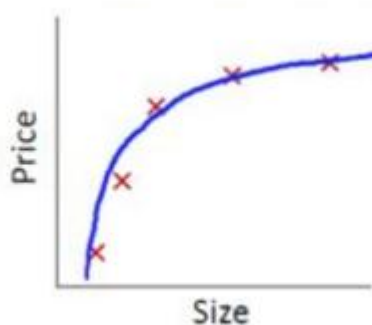


机器学习的特征构造

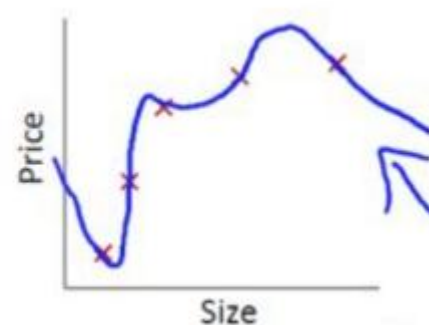
如何用线性回归模型拟合非线性关系



→ $\theta_0 + \theta_1 x$



→ $\theta_0 + \theta_1 x + \theta_2 x^2$



→ $\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

机器学习的评估

- 对数据的评估有多种方式：
- 我们会选择一部分数据作为测试集，比如20%或者10%



Training
80%

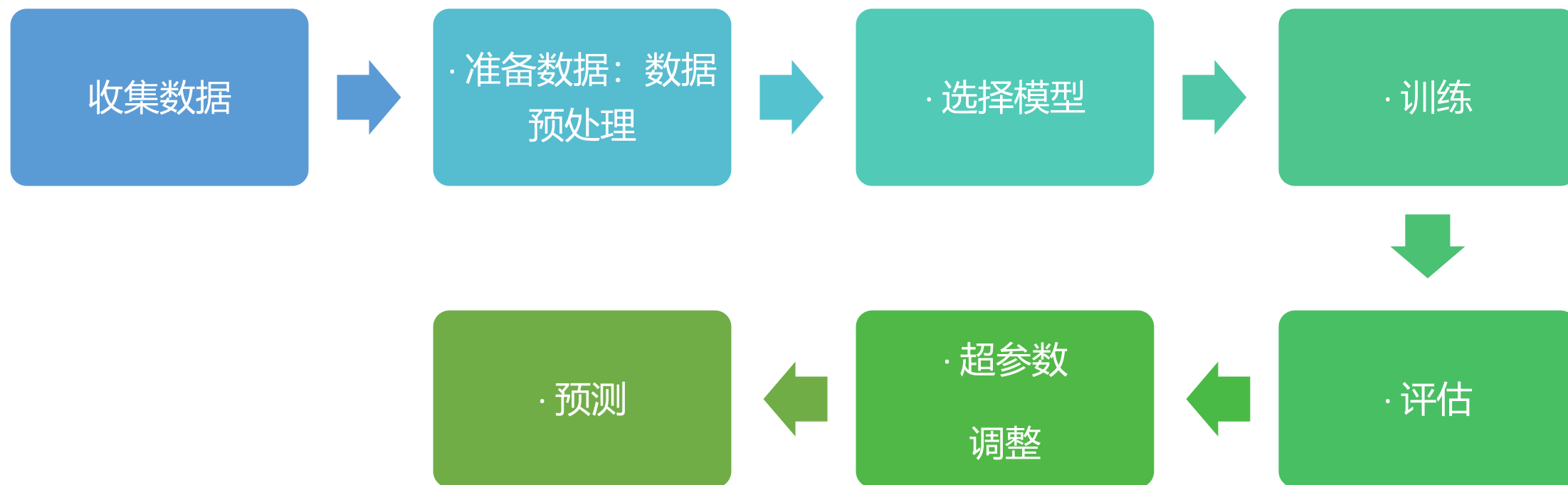


Evaluation
20%

超参数调整

- 我们还可以对模型中的参数进行调整，比如epoch的次数，学习率等
- 这些参数通常被称为超参数。调整超参数的过程比起科学更像是艺术。这是实验性的过程，并很大程度上取决于具体的数据集、模型和训练过程

Summary



CASE: XGBoost 涨跌预测

CASE: XGBoost 涨跌预测

TO DO: 搭建XGBoost 涨跌预测

1) 标签构造:

```
# 次日收盘价 > 今日收盘价 => 1 (涨) , 否则 => 0 (跌)
future_ret = df['close'].shift(-1) / df['close'] - 1
label = (future_ret > 0).astype(int)
```

这是简单的二分类标签。涨跌大致各占 50%，是一个平衡分类问题。

2) 特征输入

52 个技术因子经过 MAD 去极值 + Z-score 标准化后输入模型。

所有因子在同一尺度上（均值 0，标准差 1），避免量纲差异影响模型。

CASE: XGBoost 涨跌预测

3) XGBoost 模型

XGBoost (eXtreme Gradient Boosting) 是集成树模型的代表:

```
import xgboost as xgb
model = xgb.XGBClassifier(
    n_estimators=200,    # 200 棵树
    max_depth=5,        # 每棵树最大深度 5
    learning_rate=0.05,  # 学习率 (步长)
    subsample=0.8,      # 每棵树使用 80% 样本
    colsample_bytree=0.8, # 每棵树使用 80% 特征
    min_child_weight=10, # 叶子节点最小样本权重
    reg_alpha=0.1,      # L1 正则化
    reg_lambda=1.0,     # L2 正则化
)
model.fit(X_train, y_train)
y_prob = model.predict_proba(X_test)[: , 1] # 上涨概率
```

核心思想: 梯度提升 = 逐棵树修正残差。

每棵新树专注于前面所有树预测错误的样本, 逐步降低整体误差。

CASE: XGBoost 涨跌预测

4) 滚动训练机制

金融时序数据不能随机划分训练集/测试集（会导致未来信息泄露），

必须使用滚动窗口：

时间轴：

————[训练窗口 120天]—[预测1天]—▶

↓ 滑动

————[训练窗口 120天]—[预测1天]—▶

```
pred_df = rolling_train_predict(  
    df, feature_cols, label_col='label',  
    model_type='xgboost',  
    train_days=120,      # 用过去 120 天训练  
    retrain_interval=20, # 每 20 天重新训练  
)
```

Thinking: `retrain_interval=20` 代表什么？

重训后把 `last_train_idx = i`，之后 `i+1, i+2, ...` 只要与 `last_train_idx` 的间隔还不到 20，就不重训，继续用当前 model 预测。

CASE: XGBoost 涨跌预测

5) 评估指标

指标	公式	含义
AUC	ROC 曲线下面积	模型区分涨跌的能力, 0.5=随机
Accuracy	正确预测数/总数	整体准确率
Precision	$TP/(TP+FP)$	预测涨中真涨的比例
Recall	$TP/(TP+FN)$	真涨中被预测涨的比例
F1	$2 \times P \times R / (P + R)$	Precision 和 Recall 的调和均值

基准线: 随机猜测 Accuracy $\approx$ 50%, AUC $\approx$ 0.5。

如果模型 AUC $>$ 0.52, Accuracy $>$ 52%, 就已经具备了微弱但可利用的预测能力。

CASE: XGBoost 涨跌预测

代码逻辑：

对两只股票分别执行完整的滚动预测流程：

```
def run_single_stock(stock_code, stock_name):  
    # [1] load_stock_data() 加载日K线  
    # [2] calc_features() + preprocess_features() 特征工程  
    # [3] make_labels(df, horizon=1, method='binary') 构建二分类标签  
    # [4] rolling_train_predict(model_type='xgboost', train_days=120, retrain_interval=20)  
    # [5] evaluate_classification() 计算 AUC/Accuracy/Precision/Recall/F1  
    # [6] confusion_matrix() 混淆矩阵  
    # [7] 按月分组统计准确率变化
```

最后 compare_stocks() 对两只股票的指标做横向对比。

CASE: XGBoost 涨跌预测

运行结果

整体评估指标：

股票	AUC	Accuracy	Precision	Recall	F1	样本数
贵州茅台	0.4829	0.5025	0.4564	0.4764	0.4662	603
中芯国际	0.5395	0.5423	0.5255	0.5651	0.5446	603

月度准确率波动：

股票	均值	标准差	最高	最低
贵州茅台	0.5029	0.0963	0.6842	0.3
中芯国际	0.5442	0.1292	0.8182	0.3478

CASE: XGBoost 涨跌预测

混淆矩阵（贵州茅台）：

	预测跌	预测涨
实际跌	172	156
实际涨	144	131

结论洞察

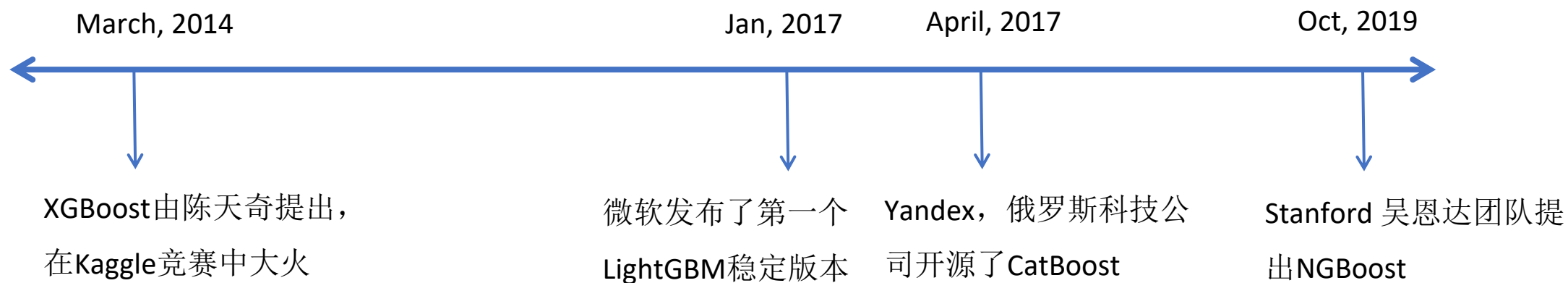
- 1. $AUC \approx 0.48$ ，接近随机基准（0.5），说明单靠 52 个技术因子对次日涨跌的预测力有限，这符合 市场弱有效 的预期
- 2. 中芯国际优于贵州茅台（Acc 0.54 vs 0.48），科技龙头的价格行为比消费龙头更有规律
- 3. 月度准确率波动很大（最高 0.81 vs 最低 0.30），说明模型在某些月份捕捉到了信号，但不稳定
- 4. 预测的核心价值不在于准确率，而在于概率值：即使 Accuracy $\approx 50\%$ ，模型输出的连续概率（0~1）仍可作为多因子选股中的一个信号维度

CASE: LightGBM 对比与调参

机器学习神器

Gradient Boosting集成学习:

- XGBoost, LightGBM, CatBoost, NGBoost实际上是对GBDT方法的不同实现, 针对同一目标、做了不同的优化处理



LightGBM

LightGBM:

- 2017年经微软推出，XGBoost的升级版
- Kaggle竞赛使用最多的模型之一，必备机器学习神器
- Light => 在大规模数据集上运行效率更高
- GBM => Gradient Boosting Machine

LightGBM

Motivation:

- 常用的机器学习算法，例如神经网络等算法，都可以以mini-batch的方式训练，训练数据的大小不会受到内存限制
- GBDT在每一次迭代的时候，都需要遍历整个训练数据多次。如果把整个训练数据装进内存则会限制训练数据的大小；如果不装进内存，反复地读写训练数据又会消耗非常大的时间。对于工业级海量的数据，普通的GBDT算法是不能满足其需求的
- LightGBM的提出是为了解决GBDT在海量数据遇到的问题，让GBDT可以更好更快地用于工业场景

数据集	XGBoost	XGBoost_approx	LightGBM
Higgs	4.853GB	4.875GB	0.822GB
Yahoo LTR	1.907GB	2.221GB	0.831GB
MS LTR	5.469GB	5.600GB	0.745GB
Expo	1.553GB	1.560GB	0.450GB

内存消耗

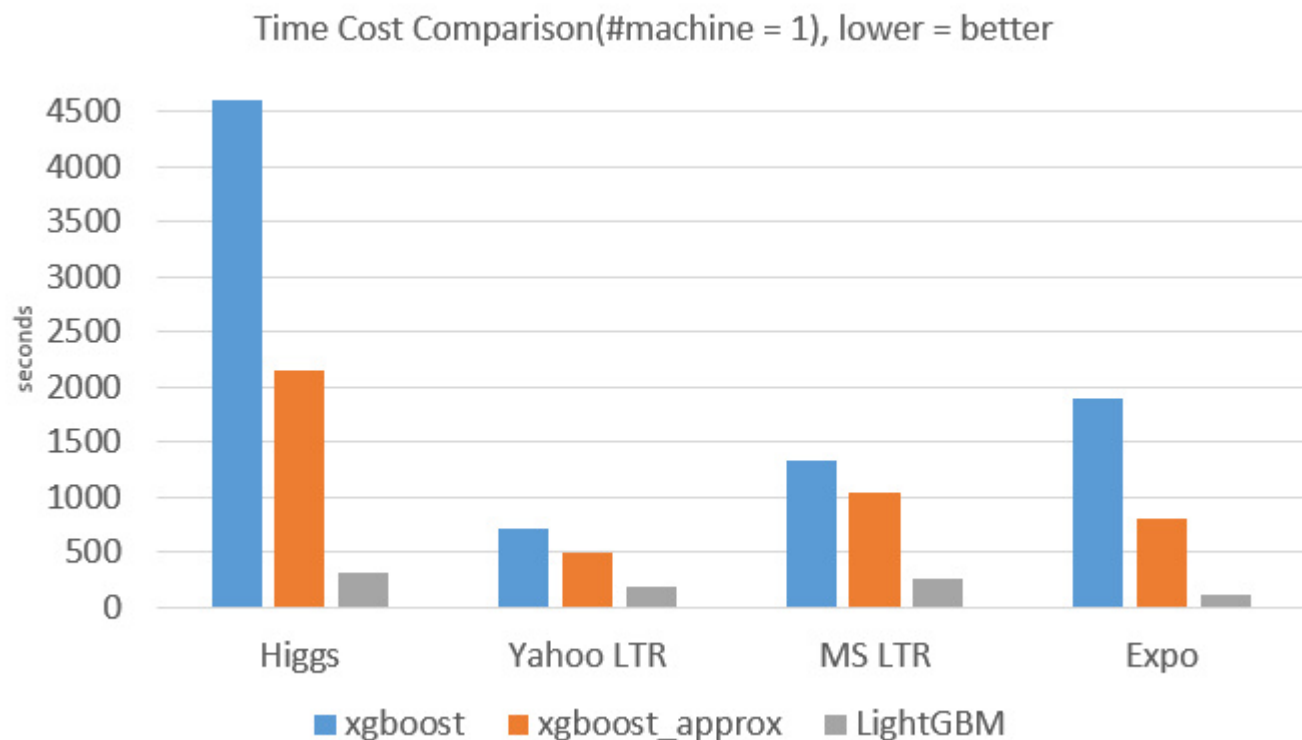
数据集	指标	XGBoost	XGBoost_approx	LightGBM
Higgs	AUC	0.839528	0.840533	0.845123
Yahoo LTR	NDCG@5	0.740316	0.739363	0.756369
MS LTR	NDCG@5	0.476245	0.474441	0.510153
Expo	AUC	0.75548	0.757071	0.781061

评估指标（AUC，NDCG）

LightGBM

LightGBM与XGBoost:

- 模型精度: 两个模型相当
- 训练速度: LightGBM训练速度更快 => 1/10
- 内存消耗: LightGBM占用内存更小 => 1/6
- 特征缺失值: 两个模型都可以自动处理特征缺失值
- 分类特征: XGBoost不支持类别特征, 需要对其进行OneHot编码, 而LightGBM支持分类特征



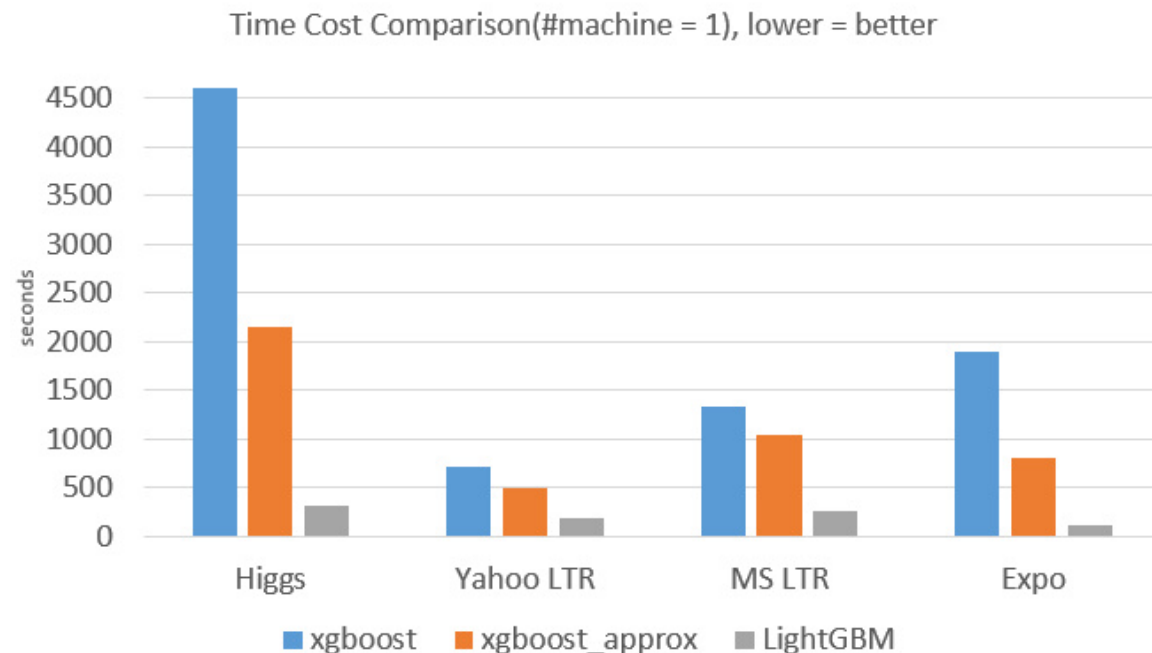
LightGBM

XGBoost模型的复杂度:

- 模型复杂度 = 树的棵数 $\times$ 每棵树的叶子数量 $\times$ 每片叶子生成复杂度
- 每片叶子生成复杂度 = 特征数量 $\times$ 候选分裂点数量 $\times$ 样本的数量

针对XGBoost的优化:

- Histogram算法, 直方图算法 => 减少候选分裂点数量
- GOSS算法, 基于梯度的单边采样算法 => 减少样本的数量
- EFB算法, 互斥特征捆绑算法 => 减少特征的数量
- LightGBM = XGBoost + Histogram + GOSS + EFB



CASE: LightGBM 对比与调参

```
import lightgbm as lgb

model = lgb.LGBMClassifier(
    n_estimators=200,
    max_depth=5,
    learning_rate=0.05,
    num_leaves=31,      # 叶子节点数 (Leaf-wise 关键参数)
    subsample=0.8,
    colsample_bytree=0.8,
    min_child_samples=20, # 叶子最小样本数
    reg_alpha=0.1,
    reg_lambda=1.0,
)
```

CASE: LightGBM 对比与调参

Thinking: 手动调参和自动调参的核心区别是什么?

手动调参 = 随机尝试 + 经验直觉, 效率极低。

Optuna 使用 TPE (Tree-structured Parzen Estimator) 贝叶斯优化:

第1步: 随机采样几组参数, 评估效果

第2步: 根据历史结果, 建立 参数→效果 的概率模型

第3步: 概率模型推荐最有希望的参数组合

第4步: 评估新参数, 更新概率模型

重复步骤 2~4, 50 次迭代

CASE: LightGBM 对比与调参

```
import optuna

def objective(trial):
    params = {
        'num_leaves': trial.suggest_int('num_leaves', 15, 63),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.15, log=True),
        'max_depth': trial.suggest_int('max_depth', 3, 8),
        'min_child_samples': trial.suggest_int('min_child_samples', 10, 50),
        'subsample': trial.suggest_float('subsample', 0.6, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.6, 1.0),
        'reg_alpha': trial.suggest_float('reg_alpha', 0.0, 1.0),
        'reg_lambda': trial.suggest_float('reg_lambda', 0.0, 1.0),
    }

    # 目标: 最大化 Purged K-Fold AUC

    fold_results = purged_kfold_cv(X, y, model_type='lightgbm', params=params)

    return np.mean([f['auc'] for f in fold_results])

study = optuna.create_study(direction='maximize', sampler=optuna.samplers.TPESampler(seed=42))
study.optimize(objective, n_trials=50)
best_params = study.best_params
```

Purged K-Fold 时序交叉验证

普通 K-Fold 随机划分数据，在时序场景下会造成未来信息泄露。

Purged K-Fold 通过在训练集和验证集之间留出 gap 天间隔来防止泄露：

普通 K-Fold（有泄露风险）：

Fold 1: [验证] [训练] [训练] [训练] [训练]

训练集包含验证集之后的数据 → 未来信息泄露！

Purged K-Fold（安全）：

Fold 1: [训练训练训练] ~~gap~~ [验证验证] [未使用]

Fold 2: [训练训练训练训练训练] ~~gap~~ [验证验证]

gap 保证训练集和验证集之间无信息泄露

```
fold_results = purged_kfold_cv(  
    X, y,  
    model_type='lightgbm',  
    n_splits=5, # 5 折  
    gap=5, # 间隔 5 天  
    params=best_params,  
)
```

CASE: LightGBM 对比与调参

脚本分 7 个部分，逐步对比实验：

- 数据准备：prepare_data() 加载茅台+中芯国际，计算特征+标签
- LightGBM 默认参数：run_rolling(model_type='lightgbm') 滚动预测
- XGBoost 对比基准：run_rolling(model_type='xgboost') 滚动预测
- Optuna 超参搜索：optuna_lgb_search(n_trials=50)，目标函数为 Purged K-Fold AUC
- 最优参数滚动预测：用 Optuna 找到的参数重新 run_rolling()
- Purged K-Fold 详情：purged_kfold_cv(n_splits=5, gap=5) 各折评估
- 性能对比表：汇总三种配置的 AUC/Accuracy/F1/耗时

Optuna 搜索空间包含 8 个超参数：num_leaves、learning_rate、max_depth、min_child_samples、subsample、colsample_bytree、reg_alpha、reg_lambda。

CASE: LightGBM 对比与调参

Optuna 搜索结果 (50 次试验, 耗时 21.8s) :

参数	最优值
num_leaves	39
learning_rate	0.0144
max_depth	6
min_child_samples	24
subsample	0.81
colsample_bytree	0.996
reg_alpha	0.451
reg_lambda	0.27

最优 Purged K-Fold AUC: 0.5683

CASE: LightGBM 对比与调参

三种模型性能对比:

股票	模型	AUC	Accuracy	F1	耗时(s)
贵州茅台	XGBoost	0.4829	0.5025	0.4662	4
	LightGBM(默认)	0.4698	0.4859	0.4343	3.7
	LightGBM(调优)	0.4791	0.5075	0.453	2.9
中芯国际	XGBoost	0.5395	0.5423	0.5446	3.5
	LightGBM(默认)	0.5536	0.5638	0.5458	2.9
	LightGBM(调优)	0.5587	0.5655	0.5529	2.9

平均 AUC 汇总:

- XGBoost: 0.4813
- LightGBM(默认): 0.4741
- LightGBM(调优): 0.4828

CASE: LightGBM 对比与调参

Purged K-Fold 各折详情（贵州茅台，LightGBM 调优）：

Fold	AUC	Accuracy	F1
1	0.607	0.5764	0.6164
2	0.5594	0.5625	0.5191
3	0.5643	0.5486	0.4628
4	0.5426	0.4861	0.0263

结论洞察：

- 1. Optuna 调优有效：LightGBM AUC 从默认 0.4741 提升到调优 0.4828，与 XGBoost 持平
- 2. 在小数据集上 LightGBM 与 XGBoost 速度接近（均约 3s），LightGBM 的速度优势在大数据集上更明显
- 3. Purged K-Fold 各折 AUC 波动较大（0.54 ~ 0.61），第 4 折 F1 仅 0.03，说明模型在某些时段泛化不足
- 4. 实战建议：使用 LightGBM 作为首选模型（速度快、内存省），配合 Optuna 定期调参以适应市场变化

Summary

1) 传统规则策略

ML 概率策略

IF MACD金叉 THEN 买 $\rightarrow P(\text{涨}) = \text{XGBoost}(52\text{个因子}) = 0.73$

单指标 / 是否决策 $\rightarrow$ 多因子 / 连续概率 / 自适应

2) 工业级流水线:

OHLCV $\rightarrow$ calc_features(52因子) $\rightarrow$ MAD去极值 $\rightarrow$ Z-score $\rightarrow$ [中性化] $\rightarrow$ 模型

3) 模型对比:

XGBoost: 经典稳定, Level-wise, 华泰研究的主力模型

LightGBM: 速度更快, Leaf-wise + GOSS + EFB, 生产环境首选

Optuna: 贝叶斯超参搜索, 50次试验自动找最优参数

Purged K-Fold: 时序安全的交叉验证, 防止未来信息泄露

打卡：股票涨跌预测



针对你的股票池，进行股票次日涨跌预测：

- 使用XGBoost 或者 LightGBM
- 提取股票的52个技术因子 + 4个基本面因子
- 构建 target，即 次日涨跌 或者 5日后涨跌
- 使用 Purged K-Fold 对验证集的未来数据进行验证

涨跌预测器准确率如何？你觉得有哪些方式，可以提升预测的准确率

股票预测：XGBR

Thinking: 刚才我们做了股票的次日涨跌预测, 还可以使用XGBoost 进行股票哪方面的预测?

- XGBoost 回归 (XGBR)

标签设计: 个股相对中证500的超额收益率 (连续值)

结果表现: 整体表现更好, 信息比率稳定, 风险控制好

=> 推荐做法

股票预测：XGBC

- XGBoost 分类 (XGBC)

标签设计：前30%标记为涨（1），后30%标记为跌（0）

结果表现：收益指标略高，但风险指标（回撤、Calmar）不如回归

⇒需要离散信号时，可以使用

分类模型输出非买即卖的明确决策（如 1=买入，0=卖出），而非具体配多少仓。

=> 适用于高频交易或严格风控场景（如日内T+0、止损线强制），需要清晰规则触发交易，而非连续权重

股票预测：XGBR-Sharpe

- XGBR-Sharpe:

标签设计：以夏普比率作为标签

结果表现：夏普比率比基础XGBR更高

=> 追求风险调整收益

XGBR-Sharpe 是一种标签工程技巧

普通 XGBR：模型学习 $y = \text{未来收益率}$ （越高越好）

XGBR-Sharpe：模型学习 $y = \text{未来收益率} / \text{未来波动率}$ （越高越好）

普通模型：预测哪只股票涨得多 → 容易选到暴涨暴跌的妖股

Sharpe 模型：预测哪只股票涨得稳 → 优选 涨得多且波动小 的标的

这种策略使用不是很多，但是适合低波动策略：
如银行理财资金、养老金，要求绝对收益稳定，
宁可牺牲部分收益也要控制回撤

股票预测：XGBR-IR

- XGBR-IR:

标签设计：以信息比率作为标签

结果表现：信息比率更高，但跟踪误差稍大

=> 追求稳定超额

专为指数增强（指增）策略设计的标签工程：模型不预测 涨多少，而是预测 能稳定跑赢基准多少。

信息比率 (IR) = 股票超额收益 (相对沪深300/中证500) / 跟踪误差 (超额收益的波动)

普通 XGBR：预测哪只股票涨得多（绝对收益导向）

XGBR-IR：预测哪只股票能稳定跑赢指数（相对收益导向）

这种策略在指数增强策略中很常见，但整体不如基础回归普及。

股票预测：XGBR-IR

- XGBR-IR：

使用场景	普及度	原因
公募指增产品	很强	考核核心就是 IR（信息比率）和超额夏普，必须用 IR 优化
私募量化选股	一般	部分会用，但更多用基础回归+事后风控控制跟踪误差
绝对收益策略	少	不太用，因为绝对收益策略不关心相对基准的表现

如果你做中证500指数增强，XGBR-IR 是核心武器；

如果你做量化对冲或多空策略，基础 XGBR 更通用。

股票预测：XGBR-Combine

XGBR-Combine：

标签设计：多标签模型集成（等权融合上述模型）

结果表现：表现最全面，随机性干扰最小

=> 最佳实践

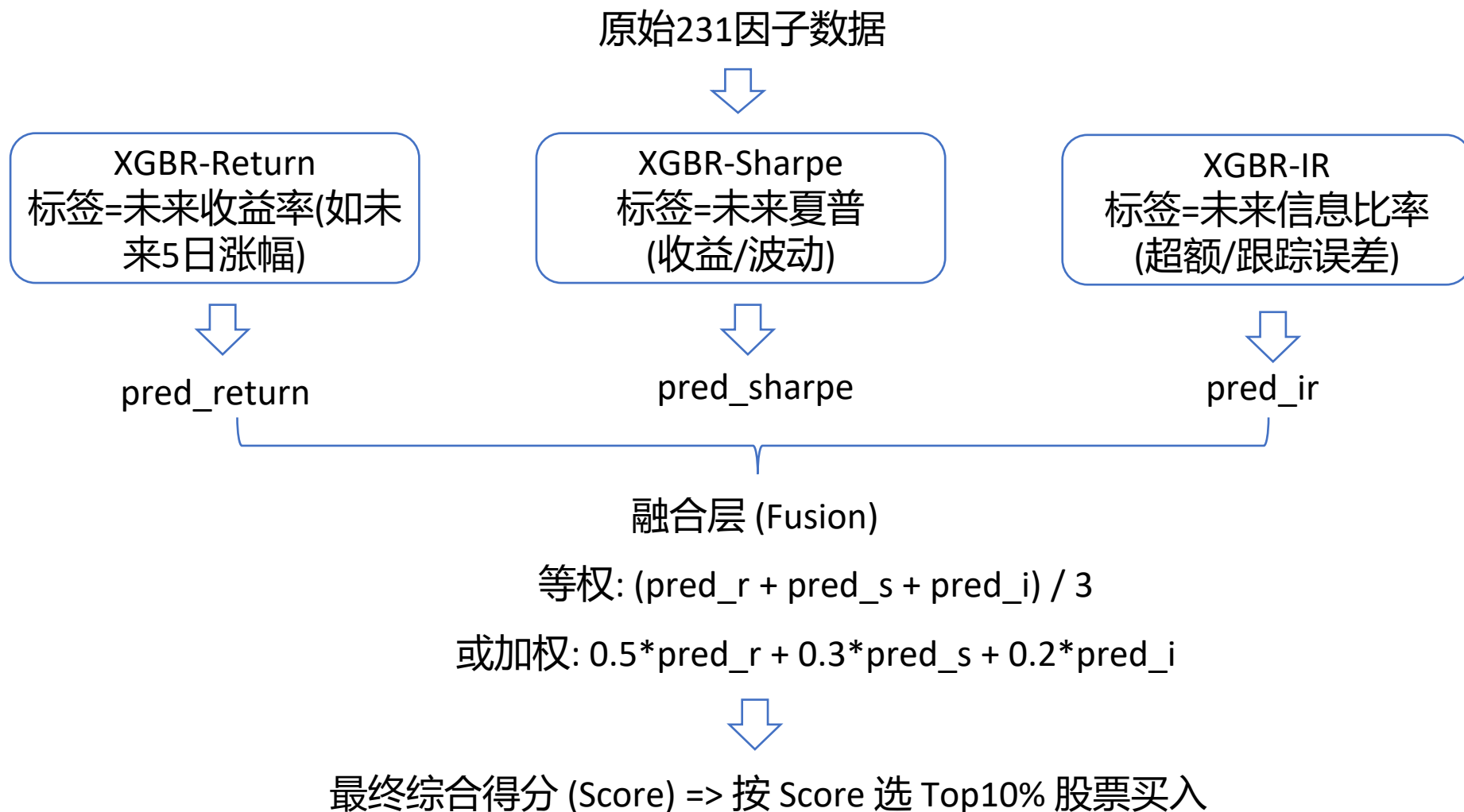
XGBR-Combine 就是不把鸡蛋放在一个篮子里的模型集成策略：

同时训练多个 XGBoost 模型，每个模型用不同的标签：

- 一个学 收益率
- 一个学 夏普比率
- 一个学 信息比率

=> 最后把它们的预测结果取平均作为最终信号。

股票预测: XGBR-Combine



股票预测：XGBR-Combine

Thinking: 为什么这样做?

每个标签都有盲区:

- 纯收益标签 (XGBR) : 可能选中 暴涨暴跌 的妖股, 回撤大
- 夏普标签 (XGBR-Sharpe) : 可能过度保守, 错过高收益机会
- IR 标签 (XGBR-IR) : 只关注相对指数的超额, 在熊市中可能"跑赢指数但还是亏钱"

集成逻辑: 就像投资分散风险, 模型也要分散标签偏见。

三个模型同时看好一只股票, 说明它既收益高、又风险低、还能跑赢基准, 比单一维度判断更可靠。

股票预测：XGBR-Combine

假设某天两个股票的因子输入相同，三个模型打分如下：

模型	贵州茅台预测值	宁德时代预测值	说明
XGBR-Return	5日涨 3%	5日涨 8%	宁德收益高，但波动大
XGBR-Sharpe	Sharpe=1.2	Sharpe=0.6	茅台风险调整收益更好
XGBR-IR (相对中证500)	IR=0.8	IR=0.5	茅台超额收益更稳定

融合计算（Rank 法）：

1. 转排名：假设当天全市场 5000 只股票

茅台：Return 排名 60% 分位，Sharpe 排名 90% 分位，IR 排名 85% 分位

宁德：Return 排名 95% 分位，Sharpe 排名 40% 分位，IR 排名 50% 分位

2. 等权融合：

茅台综合分 = $(0.6 + 0.9 + 0.85) / 3 = 0.783$ ，宁德综合分 = $(0.95 + 0.4 + 0.5) / 3 = 0.617$

选股结果：虽然宁德单看收益高，但综合风险调整后，茅台进入 Top10%，宁德被排除。



Thank You
Using data to solve problems